

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

单调队列：滑动窗口的最值利器

Monotonic Queue

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 11 月 2 日

目录

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

1. 什么是单调队列
2. 单调队列的原理与代码
3. 经典例题
4. 总结



目录

1 什么是单调队列

2 单调队列的原理与代码

3 经典例题

4 总结

单调队列的直观理解

单调队列：一种可以在队首和队尾两端进行出队操作的队列，并且队列内的元素始终保持单调递增或单调递减。

生活中的类比：**窗口排队**

- 有一个固定大小的窗口，人们依次走过窗口。
- 你想随时知道当前窗口内**身高最高的人**是谁。
- 如果队伍里来了一个更高的人，那之前比他矮且比他先来的人，**永远不可能成为最高**了，可以直接踢出队伍。
- 这样队伍的高度永远是从前到后递减的。

排队打饭时，后面来了个更强壮的人，前面弱小的就失去竞争力了

单调队列能解决什么问题

核心题型：求滑动窗口内的最大值或最小值。

常见应用

- 固定窗口大小 k 的区间最值查询。
- 优化动态规划（比如多重背包的单调队列优化）。
- 不限定的变长滑动窗口最值问题。

暴力解法的痛点

- 窗口大小为 k，每次移动都遍历一遍窗口，时间复杂度为 $O(nk)$。
- 如果 $n = 10^6$，$k = 10^5$，暴力直接超时。
- 单调队列利用“过期出队”和“淘汰劣质元素”，让每个元素只进出队一次，优化到 $O(n)$。

核心思想：剔除已经没用的数据，时刻保持队伍的精锐

目录

1 什么是单调队列

2 单调队列的原理与代码

3 经典例题

4 总结

原理：如何维护单调性

以“求滑动窗口最大值”为例：假设窗口大小为 k。

维护一个单调递减的双端队列

- 队列内存放的是数组的**下标**（不是数值，为了判断是否过期）。
- **队首过期**：如果队首下标 $\leq i-k$，说明它已经滑出窗口，弹出。
- **队尾淘汰**：如果当前 $a[i]$ 大于等于队尾的值，说明队尾元素永远不会是最大值了，弹出。
- 重复淘汰队尾，直到队列为空或 $a[i]$ 小于队尾元素。
- 将当前下标 i 加入队尾。

队首永远是目前窗口内的最大值下标

单调队列的通用代码框架

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
vector<int> maxSlidingWindow(vector<int>& a, int k) {
    int n = a.size();
    vector<int> ans;
    deque<int> q;  // 存下标

    for (int i = 0; i < n; i++) {
        // 1. 检查队首是否过期（滑出窗口）*
        if (!q.empty() && q.front() <= i - k) {
            q.pop_front();
        }
        // 2. 维护单调性：剔除劣质元素 *
        while (!q.empty() && a[q.back()] <= a[i]) {
            q.pop_back();
        }
        // 3. 当前元素入队
        q.push_back(i);
        // 4. 当窗口长度达到 k 时，记录答案
        if (i >= k - 1) {
```

单调队列的通用代码框架

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19              ans.push_back(a[q.front()]);
20          }
21      }
22      return ans;
23  }
```

目录

1 什么是单调队列

2 单调队列的原理与代码

3 经典例题

4 总结

例题一：滑动窗口最大值

题目：给定一个整数数组 `nums` 和一个大小为 k 的滑动窗口。窗口每次向右移动一位，请返回每个窗口中的最大值。

示例

- 输入：`nums = [1, 3, -1, -3, 5, 3, 6, 7]`，$k = 3$
- 输出：`[3, 3, 5, 5, 6, 7]`
- 解释：窗口 [1,3,-1] 最大值 3；[3,-1,-3] 最大值 3；[-1,-3,5] 最大值 5。

滑动窗口最大值：解法

核心思路：使用双端队列 `deque` 维护一个单调递减的队列，存入数组下标。队首始终是当前窗口的最大值下标。

关键步骤

- 第一步：判断队首是否过期。如果 `q.front() <= i - k`，就 `pop_front()`。
- 第二步：维护递减性。如果 `a[i]` 大于等于队尾的值，就不断 `pop_back()`。
- 第三步：把当前下标 i 压入队尾。
- 第四步：如果窗口已经形成（$i \geq k-1$），取出队首元素对应的值计入答案。

先查过期，再删弱者，最后入队，取队首为答案

滑动窗口最大值：代码


```cpp
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> q;  // 存下标，保证 nums[q] 单调递减
    vector<int> ans;

    for (int i = 0; i < nums.size(); i++) {
        // 1. 队首超出滑动窗口范围，过期出队 *
        if (!q.empty() && q.front() < i - k + 1) {
            q.pop_front();
        }
        // 2. 当前元素比队尾大，队尾元素失去竞争力 *
        while (!q.empty() && nums[q.back()] <= nums[i]) {
            q.pop_back();
        }
        // 3. 当前元素入队
        q.push_back(i);
        // 4. 记录当前窗口的最大值
        if (i >= k - 1) {
            ans.push_back(nums[q.front()]);
```

滑动窗口最大值：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19            }
20        }
21        return ans;
22    }
```

例题二：求连续 m 个数的最大值

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题目：给定一个长度为 n 的整数序列，以及一个整数 m。请输出序列中每一个长度为 m 的连续子序列的最大值。

示例

- 输入：$n = 5$，$m = 3$，序列为 `[1, 4, 2, 5, 3]`
- 输出：`[4, 5, 5]`
- 解释：
  - 窗口 [1, 4, 2]，最大值为 4
  - 窗口 [4, 2, 5]，最大值为 5
  - 窗口 [2, 5, 3]，最大值为 5

求连续 m 个数的最大值：解法

核心思路：与滑动窗口最大值完全一致，依然是维护一个存下标的双端队列，保证队列内的数值单调递减。

关键步骤

- 第一步：遍历数组，下标从 1 开始（或者 0 开始，注意窗口边界的判断）。
- 第二步：判断队首是否过期。如果 $i - q.front() \geq m$，则过期出队。
- 第三步：如果当前元素大于等于队尾元素，队尾出队。
- 第四步：当前下标 i 入队。
- 第五步：如果 $i \geq m$，输出队首对应的数值。

边界条件的差异：判断过期的方式与 k 的定义有关

求连续 m 个数的最大值：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
#include <iostream>
#include <deque>
#include <vector>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    deque<int> q;
    for (int i = 1; i <= n; i++) {
        // 1. 队首过期判定：距离超过 m *
        while (!q.empty() && i - q.front() >= m) {
            q.pop_front();
        }
        // 2. 维护单调递减 *
```

求连续 m 个数的最大值：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19          while (!q.empty() && a[q.back()] <= a[i]) {
20              q.pop_back();
21          }
22          q.push_back(i);
23
24          // 3. 输出结果
25          if (i >= m) {
26              cout << a[q.front()] << " ";
27          }
28      }
29      return 0;
30  }
```

目录

1 什么是单调队列

2 单调队列的原理与代码

3 经典例题

4 总结



总结：单调队列 vs 单调栈

对比项	单调栈	单调队列
操作端	只能在一端（栈顶）操作	两端操作（队首出队，队尾入队）
维护目标	寻找下一个更大/更小元素	固定窗口的最大值/最小值
过期机制	不涉及区间，无过期概念	需判断队首是否超出窗口范围
典型应用	柱状图最大矩形、接雨水	滑动窗口最值、多重背包优化

相似之处

- 本质都是用单调性剔除无用元素。
- 都存储下标而不是数值，方便判断边界和距离。
- 都是 $O(n)$ 时间复杂度。

总结：常见易错点

- **存值还是存下标**：存下标。必须通过下标判断是否过期（超出滑动窗口）。
- **等号问题**：如果求最大值，弹出队尾的条件是 `a[i] >= a[q.back()]`。如果带等号，说明队列里存的是最新的相等元素，不写等号也可以，但带等号能避免队列里堆积一堆相等的无用元素。
- **过期判断的顺序**：判断过期一定要在最前面写，否则可能把原本已经过期、但还没从队尾弹出的元素误认为队首。
- **窗口形成条件**：通常当 $i \geq k-1$ 时才开始记录答案，因为此时窗口大小才刚好为 k。

核心口诀：过期出队，弱者出队，强则入队，取队首

附录：单调队列求最小值模板


```cpp
// 求长度为 k 的滑动窗口内的最小值
vector<int> minSlidingWindow(vector<int>& a, int k) {
    int n = a.size();
    vector<int> ans;
    deque<int> q;

    for (int i = 0; i < n; i++) {
        // 1. 判断过期 *
        if (!q.empty() && q.front() <= i - k) {
            q.pop_front();
        }
        // 2. 维护单调递增（求最小值，遇到更小的，前面大的就弹出）*
        while (!q.empty() && a[q.back()] >= a[i]) {
            q.pop_back();
        }
        q.push_back(i);

        // 3. 记录答案
```

附录：单调队列求最小值模板


```
19          if (i >= k - 1) {
20              ans.push_back(a[q.front()]);
21          }
22      }
23      return ans;
24  }
```